

# RETHINKING TRANSFER IN CONTINUAL LEARNING: A REPLAY-BASED REALISATION

Yang Meng, Zhenya Liu, Zhuokai Zhao<sup>†</sup>, Yuxin Chen<sup>†</sup>

Department of Computer Science, University of Chicago

ymeng3@uchicago.edu

<sup>†</sup> Co-last authors.

## ABSTRACT

Continual learning studies how deployed language models can continually acquire new tasks without expensive retraining from scratch. Existing methods, whether rehearsal-based (replaying stored past data) or rehearsal-free (regularising or isolating parameters), overwhelmingly target one objective: preventing catastrophic forgetting. Forward transfer — the past helping the future — has meanwhile been pursued almost exclusively through parameter reuse, with no explicit account of when transfer should be expected at all. We begin one step earlier: before designing a transfer mechanism, we ask when transfer should exist at all. We answer with a framework of three measurable conditions: the target task must leave room for improvement beyond its own limited supervision, transferable information must survive continued optimisation, and replay must come from compatible previous tasks. We instantiate this view as Transfer-Selective Replay (TSR), which selects replay data predicted to benefit the incoming task rather than replaying past examples indiscriminately. Selection is guided by a zero-training task signature, while distillation preserves stability on previous tasks. Under the standard continual-learning protocol in the low-budget regime, TSR consistently improves forward transfer while maintaining stability, outperforming existing replay baselines across heterogeneous and homogeneous task streams. More broadly, the results argue for treating transfer as a first-class objective of continual learning, to be understood before it is engineered.

## 1 INTRODUCTION

Large language models are trained once, at enormous cost, and then deployed into a world that does not stand still: new domains, new formats, and new skills keep arriving after release (Wang et al., 2023b; 2024a; Tankala et al., 2025; Asawa et al., 2026). Retraining from scratch for every addition is prohibitive, so a deployed model must learn tasks *sequentially* — the setting of continual learning (De Lange et al., 2021; Ke & Liu, 2022; Momeni et al., 2025; Chen et al., 2026). Its defining failure mode has been known for decades: *catastrophic forgetting*, where optimising for the new task overwrites what earlier tasks built (Kirkpatrick et al., 2017; Lopez-Paz & Ranzato, 2017; Zheng et al., 2025). The field frames the resulting tension as a *stability–plasticity* trade-off: stability preserves the past, plasticity lets the model keep learning, and improving either naively costs the other (De Lange et al., 2021; Momeni & Liu, 2025).

Current continual learning methods can be broadly divided into two families. *Rehearsal-based* methods keep a small buffer of past examples and use it during new-task training — replayed directly (Chaudhry et al., 2019b; Rebuffi et al., 2017), with logit anchoring (Buzzega et al., 2020), selected for retention (Aljundi et al., 2019a;b), or as gradient constraints (Chaudhry et al., 2019a). *Rehearsal-free* methods keep no data: they regularise parameters toward past solutions (Kirkpatrick et al., 2017; Li & Hoiem, 2017) or isolate tasks in separate parameter subsets — which in the LLM era usually means separate LoRA adapters (Hu et al., 2021), kept orthogonal (Wang et al., 2023a), gated (Liang et al., 2025), or organised hierarchically (Qian et al., 2025). The families differ in

Code is available at <https://github.com/ymeng3/transfer-selective-replay>.

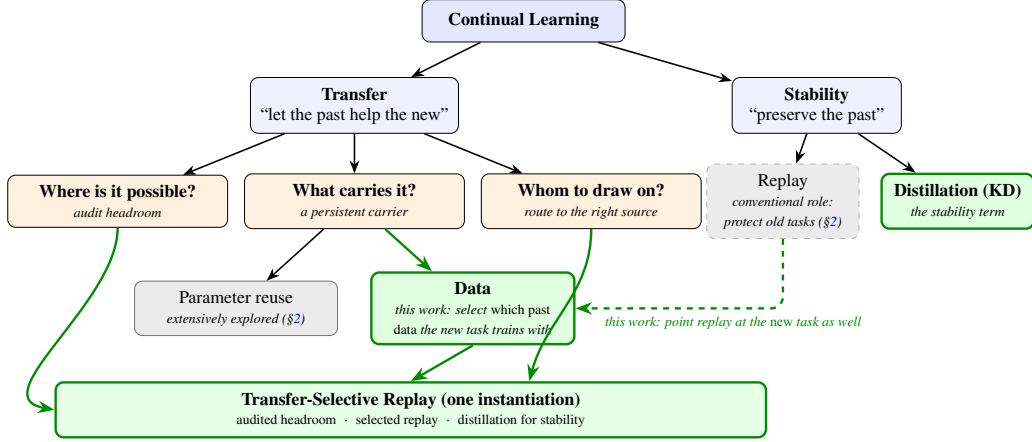


Figure 1: **A framework for continual transfer, and one instantiation.** Continual learning serves two objectives. On the transfer side we organise the design space around three questions — where transfer is possible (headroom), what carries it (parameters vs. data), and whom to draw on (source selection); prior transfer mechanisms occupy the parameter branch (§2), and this work takes the data branch, turning transfer into a data-selection problem. On the stability side, replay’s conventional role is protecting old tasks; we point the same machinery at the *new* task as well (dashed arrow) and assign the stability term to distillation (§5.3). Transfer-Selective Replay instantiates all three answers (§5).

machinery but agree on the objective: both answer the stability question, *how do we keep the past from being lost?*

The complementary question — whether past tasks can help learning future ones — has received far less attention. *Forward transfer*, the improvement in learning a new task due to knowledge acquired from previous tasks, is the promised upside of continual learning; it has been measured since GEM (Lopez-Paz & Ranzato, 2017) and designed for since Progressive Networks (Rusu et al., 2016). Yet on modern LLM benchmarks it is routinely reported as absent or negative. Existing attempts to promote forward transfer have relied almost exclusively on *parameter reuse*: lateral connections and module reuse (Rusu et al., 2016; Veniat et al., 2021), task masks and shared adapters (Ke et al., 2020; 2021a), prompt reuse (Qin & Joty, 2022), and, for LLMs, routing, initialising from, or merging past adapters (Huang et al., 2023; Ilharco et al., 2023; Qian et al., 2025). The assumption behind this line is so uniform that it is rarely stated: *past knowledge lives in past parameters*. Past-task *data*, meanwhile, has been used almost exclusively to prevent forgetting rather than to improve learning on new tasks. Replay has been developed to preserve old-task performance, while forward transfer has relied on parameter reuse; the two directions have remained largely separate. And although negative transfer is increasingly documented (Choi & Kim, 2026; Chen et al., 2026), the field has no explicit formulation of *when* transfer should be expected at all.

This paper starts one step earlier: *before designing a transfer mechanism, we ask what makes transfer possible*. We study the question in *low-budget* continual learning — each task arrives with a fixed, small training sample (Zhao et al., 2024). This is the continual-learning regime where forward transfer becomes operationally valuable: with plentiful target data a task saturates on its own and leaves nothing for the past to supply (§4.1). Under a fixed budget, the value of transfer is not to replace target supervision but to improve the performance attainable from that budget by exploiting previously acquired experience.

We identify three measurable conditions that determine whether transfer is possible (§4):

- *Where can transfer happen?* Only where the target task leaves headroom beyond what its own limited supervision can achieve — an auditable property before any transfer mechanism is introduced (§4.1).
- *What carries transfer?* Only a persistent carrier. Replay data repeatedly reinforces the same optimisation direction throughout training, whereas one-shot parameter reuse is progressively washed out (§4.2).

- *Whom should the new task draw on?* Source quality varies sharply, but compatible sources can be identified at zero training cost using a label-aware *task signature* (§4.3).

We organise the design space of continual transfer around these three questions (Figure 1).

**Transfer-Selective Replay (TSR)** is one *instantiation* of this framework. Choosing replay data as the carrier turns forward transfer into a *data-selection* problem: the central question is no longer whether to replay, but which previous experience should be replayed for the incoming task. Concretely, TSR maintains a task memory holding one record per past task — a task signature, a small data subsample, and the adapter snapshot from when that task ended. When a new task arrives, a few forward-backward passes at a fixed shared initialisation produce its signature at zero training cost; a softmax over signature similarities then routes every replay batch to the past data most aligned with the new task, and each replayed batch is distilled against its own era’s snapshot. **Distillation** complements replay by preserving stability, allowing replay to focus on transfer — a role our analysis later identifies as optimisation regularisation rather than knowledge preservation (§5.3).

We evaluate TSR under the standard continual-learning protocol on TRACE-8 (Wang et al., 2023b) and NumGLUE-8 (Mishra et al., 2022), across three backbone families (Qwen2.5 (Qwen Team, 2024), Gemma 3 (Gemma Team, 2025), and Llama 3.2 (Grattafiori et al., 2024)) and thirteen state-of-the-art continual-learning methods spanning both rehearsal-based and rehearsal-free approaches. TSR consistently outperforms existing replay baselines by improving forward transfer while maintaining positive backward transfer.

### Contributions.

1. **A framework for continual transfer.** Three questions — where (headroom), what (carrier), whom (source) — each made measurable: benchmark headroom audits with oracle ceilings before mechanism design, a same-benchmark carrier dissociation, and the first pairwise data-transfer matrix in continual learning with LLMs, including its negative cells (§4).
2. **An instantiation that redefines the role of replay.** The field’s most reliable stability tool serves both objectives, and had so far been assigned only one: TSR points replay at the *current* task, routed by a zero-training, label-aware signature over a task memory of (signature, data, snapshot) records. Its selection matches the hindsight-oracle ceiling, and it is the strongest replay policy under the standard protocol (§5–6).
3. **A clarification of the role of distillation in continual LLM learning.** Through a systematic teacher audit — identity, content, loss shaping, timing, snapshot source — we show that distillation contributes through optimisation *anchoring* rather than transferable teacher knowledge: its teachers matter as anchors, not as knowledge sources. This dissociates the field’s knowledge-preservation narrative from the operational mechanism, and motivates assigning distillation exclusively to the stability objective (§5.3, Appendix B.4).

## 2 RELATED WORK AND POSITIONING

**Forward transfer.** Transfer as an explicit goal of continual learning is not new: Lopez-Paz & Ranzato (2017) formalised forward and backward transfer metrics, and designing for forward transfer dates to Progressive Networks (Rusu et al., 2016). GEM’s forward transfer is a zero-shot quantity (test before training); the training-time sense studied here — earlier tasks improving the *learning* of the current one — is the promised upside of sequential learning, and reusing source data for it is well studied outside CL as intermediate-task training (STILTs; Phang et al., 2018; Vu et al., 2020) without ever being instantiated inside a continual stream. What is missing is therefore not the objective but a *formulation*: an account of when transfer should exist, and how to measure it, before a mechanism is designed (§4).

**Parameter-side transfer.** The mechanisms proposed to realise transfer during training are almost exclusively parameter-side: module reuse (Veniat et al., 2021), masks and adapters over a shared backbone (CAT/B-CL/CTR; Ke et al., 2020; 2021b;a), meta-learned representations (Javed & White, 2019), and soft-prompt reuse (Qin & Joty, 2022) — a framing the NLP-CL survey of Ke & Liu (2022) makes explicit by listing all transfer mechanisms as architectural. In the LLM era the same assumption drives inference-time composition (LoRAHub, task arithmetic, AdapterSoup; Huang et al., 2023;

Ilharco et al., 2023; Chronopoulou et al., 2023) and continual-training constraints: orthogonal adapters (O-LoRA/N-LoRA; Wang et al., 2023a; Yang et al., 2025), single-adapter merging (Qiao & Mahdavi, 2025), and gated or routed per-task adapters (Zhang et al., 2025; Liang et al., 2025; Ostapenko et al., 2024). These works answer the design question differently but share one assumption: parameters are the transferable object. This paper investigates the complementary data-side formulation, and §4.2 measures where each carrier survives — inference-time composition is never trained against, and nothing erases it.

**Data replay.** Replay is classically a stability mechanism (ER, DER++, iCaRL; Chaudhry et al., 2019b; Buzzega et al., 2020; Rebuffi et al., 2017), and the replay-selection literature optimises the same objective: MIR (Aljundi et al., 2019a) picks maximally interfered samples, GSS (Aljundi et al., 2019b) gradient-diverse ones. The structurally closest works remain retention-directed: MER (Riemer et al., 2019) places a generic buffer inside a transfer-maximising meta-objective without selecting *which* past task helps; Adaptive Memory Replay (Smith et al., 2024) selects past-data clusters conditioned on the current task, but its bandit reward is a forgetting metric; InsCL (Wang et al., 2024b) modulates replay by task similarity in the *inverse* direction (dissimilar tasks get more replay, to protect them); and learned replay controllers (Shihab et al., 2026) frame replay as data selection, again for retention. Replay has been used to preserve the past, not to accelerate the future — the objective this paper inverts (§5).

Pointing replay at the new task then requires knowing which past task will help — a task-similarity question. Fisher embeddings (Task2Vec; Achille et al., 2019; Vu et al., 2020), optimal-transport distances (OTDD; Alvarez-Melis & Fusi, 2020; Nguyen et al., 2025a), gradient-based task groupings (Fifty et al., 2021), and example-level gradient selection (LESS; Xia et al., 2024) all predict task relationships; concurrent work predicts transfer from dataset traits (Krishna et al., 2025) or selects task orderings (Nguyen et al., 2025b). Closest inside CL, TreeLoRA (Qian et al., 2025) routes new tasks to adapter groups by gradient similarity at *evolving* weights — the same class of signal, coupled to a parameter carrier; the carrier dissociation of §4.2 predicts where that coupling holds. The measured pairwise transfer matrix of §4.3 provides the prediction target these representations have lacked: signatures are validated against measured transfer rather than a routing heuristic.

**Distillation.** In continual learning, distillation is narrated as knowledge preservation — from LwF and iCaRL (Li & Hoiem, 2017; Rebuffi et al., 2017) to DER++’s stored logits (Buzzega et al., 2020) and recent self-distillation (Yang et al., 2024). Adjacent fields already read KL-anchoring as regularisation: reference-policy penalties in RL (Teh et al., 2017) and label-smoothing accounts of supervised distillation (Yuan et al., 2020; Mobahi et al., 2020). Missing is the continual-LLM dissociation — whether the teacher’s *knowledge*, or merely its existence as an *anchor*, produces the gains; §5.3 answers by audit.

**Negative transfer.** MERIT (Choi & Kim, 2026) detects gradient-level conflict in *parallel* multi-task tuning; OPR (Chen et al., 2026) selects replayed rollouts by quality rather than source-task identity; surveys note that replay can induce negative forward transfer in fragile domains (Wang et al., 2024a). These works document the phenomenon. Rather than documenting it, this paper predicts and avoids it: the negative cells of the transfer matrix are exactly what source selection routes around (§4.3).

### 3 PROBLEM FORMULATION

#### 3.1 CONTINUAL LEARNING

We consider a task-incremental continual learning setting: a learner faces a stream of tasks  $\mathcal{T} = (T_1, \dots, T_K)$ , arriving one at a time, where each task  $T_t$  is a distribution  $D_t$  over input–output pairs and provides a training and a test sample from it. We study continual fine-tuning with a single accumulating parameter state: the learner maintains one set of trainable parameters  $\theta$  throughout the stream; no per-task parameters are added and no task identity (i.e., which task a test example belongs to) is available at inference. When task  $T_t$  arrives, sequential learning solves

$$\theta_t = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim D_t} [\ell(f_{\theta}(x), y)], \quad (1)$$

optimised from the previous state  $\theta_{t-1}$ ; earlier tasks are accessible only through bounded experience retained from the stream (possibly none) (De Lange et al., 2021). Solving Eq. (1) naively is known to

degrade earlier tasks — catastrophic forgetting — and, less discussed, to forgo any help earlier tasks could give the current one; continual-learning methods differ in how they approximate Eq. (1) under this constraint (§5). Throughout, we study the *low-budget* regime: each incoming task provides only a limited number of labeled examples, while experience from earlier tasks is already in hand. The concrete budget and the instantiation of the parameter state are described in §6.1.

### 3.2 TRANSFER

Throughout this paper, *forward transfer* refers to the contribution of previously acquired experience to the learning of the current task, under the fixed target-data budget. Conceptually, we decompose current-task learning into two contributions:

$$\text{current-task learning} = \underbrace{\text{target-data contribution}}_{\text{what the task teaches itself}} + \underbrace{\text{transfer contribution}}_{\text{what the past supplies}}.$$

The transfer contribution is the object of this paper. It is distinct from GEM’s zero-shot forward-transfer metric (test before training) (Lopez-Paz & Ranzato, 2017) and from generalisation to unseen tasks; and it is *latent* — no standard continual-learning metric reads it off directly. Since the contribution is not directly observable, the remainder of the paper introduces measurable surrogates: §4 constructs its measurement and asks under what conditions it exists at all.

## 4 THREE CONDITIONS FOR REALIZED TRANSFER

We organize the investigation around a simple decomposition of when forward transfer is realized in a continual stream:

$$\text{transfer}(j \rightarrow t) > 0 \text{ requires } \underbrace{\text{headroom}(t)}_{\text{Where?}} \wedge \underbrace{\text{a persistent carrier}}_{\text{What?}} \wedge \underbrace{\text{the right source } j}_{\text{Whom?}}, \quad (2)$$

We examine each condition in turn; §5 then derives the method from them.

### 4.1 WHERE: HEADROOM AS TRANSFER OPPORTUNITY

Before asking how transfer can be realised, we must first ask whether any transfer opportunity exists at all.

**Condition 1** *Without benchmark headroom, transfer is unmeasurable by any mechanism: realised transfer is bounded by what additional in-domain data buys.*

**Headroom.** The transfer contribution of §3 is latent — training on mixed data yields one outcome in which target learning and transfer are entangled — so we first need a diagnostic for *where* it could appear at all. Three training runs under identical budgets — the *probe* protocol used throughout — define every quantity in the paper: **target-data** (the  $N$  target examples alone), **target-full** ( $10N$  target examples), and **mixed**( $j$ ) (a 50/50 mixture of source task  $j$ ’s data and the  $N$  target examples). Writing  $S_t(N)$  for end-of-training test accuracy after training on  $N$  of task  $t$ ’s own examples,

$$\text{headroom}_t(N) = S_t(N_{\max}) - S_t(N), \quad (3)$$

with  $N_{\max}$  a fixed well-resourced reference (ten times the standard budget in our protocol): the gain that additional *in-domain* supervision — the most valuable supervision a task can receive — would still buy at budget  $N$ .

**Interpretation.** Headroom measures how much improvement remains unavailable from the task’s own budget — *task saturation*, and through it the *transfer opportunity*. Figure 2(a) is the evidence: the target-data curve rises from .397 at  $N=10$  to .588 at  $N=500$ , so headroom is not a fixed property of a benchmark but a function of the target-data budget — large exactly where labels are scarce.

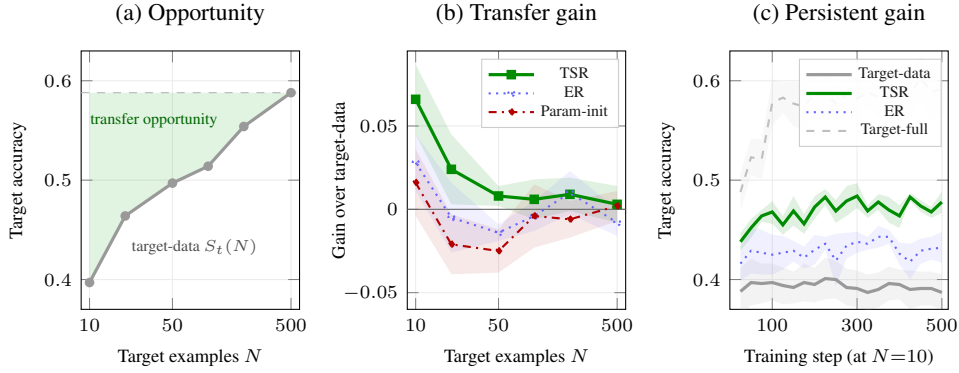


Figure 2: **Transfer opportunity shrinks with the target budget, and the gain lives where opportunity is large** (flagship target numglue-cm, the matrix’s strongest pair;  $n=5$  seeds per point; shading  $\pm 1$  s.e.m.). (a) headroom vs. budget; (b) paired transfer gain over target-data (TSR denotes selected replay, ER uniform replay); (c) at  $N=10$  the gain holds from the first evaluation to the last step. Per-target numbers: Appendix A.2.

**Quantification.** Since the contribution is latent, we report its practical effect as *equivalent target supervision* (a sample-efficiency reading): a measured gain at budget  $N$  is converted, through the  $S_t(N)$  curve, into the number of additional target labels that would buy the same improvement. This is the long-standing convention in transfer measurement — samples-to-threshold in RL transfer (Taylor & Stone, 2009), source ranking under limited target supervision in taskonomy (Zamir et al., 2018), and *effective data transferred* in neural scaling laws (Hernandez et al., 2021). Figure 2(b): at  $N=10$ , selected mixing gains  $+0.066$  — the performance of  $\approx 20$  target examples, a  $2\times$  label efficiency, 35% of the remaining opportunity — and the gain decays as saturation closes, never crossing zero on this target; across all five classification targets the same pattern holds with smaller amplitude, turning negative once headroom is spent. Panel (c) shows the gain is present from the first evaluation and sustained for all 500 steps — not an endpoint artefact (per-target numbers are in Appendix A.2). The two other arms preview the remaining conditions: uniform replay (ER) captures less than half the low-budget gain and sits below selection at every budget (§4.3); the parameter-init arm never separates from zero (§4.2).

**Implication.** A benchmark with zero headroom cannot reveal transfer, whatever the mechanism. On the canonical 15-task suite of Razdaibiedina et al. (2023) (PP15; Wang et al., 2023a; Qin & Joty, 2022), an instruction-tuned 0.5B base leaves essentially none: ten times more target data buys  $+0.02$  on average, and an audit of fourteen parameter-reuse configurations with an oracle-selected source lands within one seed standard deviation of the plain baseline (Appendix A.1). TRACE-8 (Wang et al., 2023b), by contrast, retains real headroom at the same scale — hence our benchmark choice (§4.3 draws the benchmark-design corollary). Wang et al. (2023b) argued qualitatively that classic suites lack challenge for aligned LLMs, and Akhtar et al. (2026) recommend reporting saturation statistics; the headroom audit is the actionable, budget-conditioned version. Our merge-protocol artefact — an apparent transfer signal  $35\times$  its protocol-clean value — joins the documented genus of evaluation artefacts (Kong et al., 2024; Zheng et al., 2025; Sun et al., 2025).

#### 4.2 WHAT: THE CARRIER MUST PERSIST

Headroom tells us whether transfer is *possible*; it does not tell us how the transferred signal reaches the learner. This motivates the next question: *what carries transfer*? A carrier can take many forms — parameters, data, prompts, external memory — and the stream constrains it only to be something retained from past tasks. Whatever its form, the carrier’s signal must survive the continual optimisation.

**Condition 2** *Realised transfer requires a carrier that preserves its signal throughout optimisation; a transient carrier is erased regardless of headroom or source quality.*

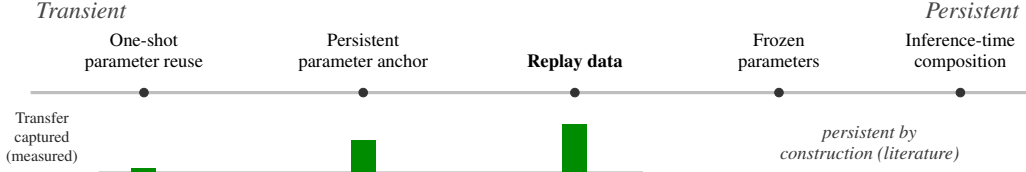


Figure 3: **Candidate carriers, ordered by the persistence of their optimization signal.** The transfer gain follows persistence rather than whether the carrier is parameters or data; bars show the measured share of the data-mixing accuracy gain each carrier captures (Appendix B.2). The literature examples shown here are persistent by construction.

| source\target | cstn         | fomc         | meet         | py15         | sciq         | ngcm         | ngds         | 20mn         |
|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| cstance       | —            | <b>−0.18</b> | −0.20        | −0.70        | +0.19        | −0.01        | −0.69        | −0.63        |
| fomc          | <b>−0.07</b> | —            | −0.24        | −0.45        | <b>−0.09</b> | +0.14        | −0.27        | −0.69        |
| meetingbank   | +0.02        | −0.02        | —            | −0.34        | +0.12        | <b>−0.26</b> | −0.53        | −0.45        |
| py150         | −0.03        | −0.15        | −0.29        | —            | +0.16        | −0.10        | −0.28        | −0.46        |
| scienceqa     | −0.04        | −0.09        | <b>−0.36</b> | <b>−0.76</b> | —            | +0.26        | −0.47        | −0.75        |
| numglue-cm    | +0.01        | −0.04        | −0.21        | −0.68        | −0.03        | —            | <b>−0.88</b> | <b>−0.85</b> |
| numglue-ds    | +0.01        | −0.11        | −0.29        | −0.62        | −0.01        | −0.20        | —            | −0.80        |
| 20minuten     | +0.06        | −0.02        | −0.18        | −0.35        | +0.27        | −0.16        | +0.00        | —            |

Table 1: Transfer matrix on TRACE-8 (pairwise joint training of source  $j$  mixed 50/50 with  $N=50$  target examples vs. target-data; mean of 5 seeds; bold = column best). **Green** = source helps the target (darker = larger loss reduction); **red** = source *hurts*.

Figure 3 tests the requirement across candidate carriers on TRACE-8: identical budgets, the same source, different carriers. A transient carrier — one-shot initialisation from the source-trained adapter — captures none of the mixing gain; its early advantage is real but erased by continued training (*wash-out*). A persistent *parameter* carrier — an L2 anchor toward the same source adapter, kept in the loss at every step — captures roughly two-thirds (Appendix B.2). Persistent *data* — the source’s examples mixed into every batch — captures all of it. The transfer gain thus rises with persistence, not with whether the carrier is parameters or data. The same axis organises the literature: frozen columns (Rusu et al., 2016) and inference-time composition (Huang et al., 2023) succeed and are persistent by construction — the source signal is never optimised away — whereas merge-then-train pipelines re-enter at the transient end; the literature appears contradictory only because it mixes transient and persistent parameter mechanisms (Appendix B.1).

Among the carriers we evaluate, replay data is the simplest persistent one: no parameter growth, no architectural change, no task identity at inference. Data is therefore not privileged because it is data, but because it is the simplest carrier whose signal is reintroduced throughout optimisation — the premise from which §5 derives the method, and which §6 stress-tests at benchmark scale.

#### 4.3 WHOM: SOURCE SELECTION

Knowing that transfer is possible, and knowing what carries it, still leaves one question open: which previous task should the learner draw on?

**Condition 3** *Given headroom and a persistent carrier, realised transfer is determined by source choice — including its sign; and the choice is predictable from label-aware learning directions at the shared initialisation.*

##### 4.3.1 THE TRANSFER MATRIX

We measure the full transfer matrix over all ordered pairs of TRACE-8 (Table 1).

**Three structural findings.** (1) *Semantic pairs are mutual best sources:* fomc  $\leftrightarrow$  cstance are each other’s best source; NumGLUE-cm is NumGLUE-ds’s best at  $-0.88$ , the largest cell. (2) *Negative transfer is real and target-specific:* cstance, scienceqa, and numglue-cm are fragile targets where

the wrong source actively hurts (worst cells +0.27, +0.26) — the empirical basis for §5’s headline. (3) *Generation targets absorb anything* (−0.2 to −0.85 from nearly all sources): pair-specific selection is a classification-task phenomenon here.

**Benchmark-design corollary.** Contrasting the two benchmarks: *a CL benchmark should report its data headroom (at a stated data budget) and transfer structure as part of its specification*. PP15-class suites evaluated with instruction-tuned bases have neither, and any transfer comparison run on them inherits a ceiling of zero regardless of method (Appendix A.1) — which may explain why routing and reuse comparisons in this literature so often report method-insensitive results.

#### 4.3.2 TASK SIGNATURES

The matrix defines what a selector must predict; measuring it requires training every ordered source–target pair — feasible for analysis, impractical in continual learning, where the source must be chosen the moment a new task arrives. We therefore seek a *task signature*: a training-free representation of each task from which measured transfer can be predicted. The candidates we evaluate span three increasingly informative families — input similarity, dataset geometry, and learning direction (Table 2). Only the *gradient signature* consistently identifies the best source:

$$g_t = \frac{1}{m} \sum_{b=1}^m \nabla_B \mathcal{L}_{CE}(b) \Big|_{(A_0, B_0)}, \quad \text{score}(j, t) = \cos(g_j, g_t), \quad (4)$$

Here  $g_t$  is the mean gradient over the  $m$  probe batches, taken at the fixed shared initialisation  $(A_0, B_0)$  and flattened across layers;  $B$  is the zero-initialised LoRA factor, so  $g_t$  is the adapter’s entire first-order learning direction, and sources are scored by the cosine between directions.

**Mechanism.** The signature is not a semantic representation: it captures the *optimisation bias* a task induces at the shared initialisation — the direction in which the task asks the adapter to change — so it reflects what a task asks the model to *learn*, not what its data looks like. This is also why alignment should predict transfer under replay. Replayed batches act on the same accumulating adapter as the target’s own batches: gradient steps from an aligned source reinforce the trajectory the target demands — the replay supplies optimisation bias rather than additional supervision — whereas steps from a conflicting source compete with the target update for the same parameters. If this interpretation is correct, aligned source updates should produce more compatible descent on the target than conflicting ones.

**Local intervention.** We test this optimisation hypothesis by intervention. Holding the adapter at the shared initialisation — the state where signatures are taken — we apply one target update, add an equal-norm gradient perturbation from a past task — so that only the replay direction, not its magnitude, differs across sources — and measure the loss change on an independent held-out target batch, isolating its effect from additional target supervision or long-horizon optimisation (protocol in Appendix A.2). Sources grouped by the stored signature produce strictly ordered marginal descent (Table 2, right), demonstrating that replay directions selected by the signature provide progressively more compatible optimisation steps. Current-gradient alignment predicts the immediate reduction within target (Spearman  $\rho = .87$ ), and the stored signature preserves this geometry ( $\rho = .85$  against current-gradient cosines) — routing on the stored signature therefore predicts compatible descent rather than input similarity; §6 tests whether this local effect survives repeated optimisation.

**Takeaways.** (i) **The gradient signature is the only reliable argmax predictor:** it identifies the oracle-best source for 4/8 targets overall and 4/5 classification targets; its one miss selects the matrix’s second-best source. (ii) **Direction, not magnitude, carries the signal:** squaring gradients (Fisher/task2vec-style) destroys it; normalising per-batch directions reproduces it exactly. (iii) **Functional beats distributional:** input-similarity and distribution-geometry signatures find the best source for at most 2/8 targets, and adding labels does not fix it (JointOT: top-1 0/8 despite ranking well) — the failures mirror the mechanism: distributional signatures ignore what tasks ask the model to *do*, and gradient magnitudes ignore *where* the demands point. Further analyses are reported in Appendix A.2.

With measurable answers to *where*, *what*, and *whom*, we now construct one algorithmic instantiation.

| Prediction   |                                    |       | Mechanism          |                     |
|--------------|------------------------------------|-------|--------------------|---------------------|
| Signature    | Signal                             | Top-1 | Added update       | marginal $\Delta L$ |
| Proto        | input similarity                   | 1/8   |                    |                     |
| HiddenOT     | distribution geometry              | 2/8   |                    |                     |
| JointOT      | label-aware distribution           | 0/8   |                    |                     |
| Fisher       | gradient magnitude                 | 1/8   | Extra target batch | −1.08               |
| GradB_last4  | gradient direction (last 4 blocks) | 3/8   | Top-aligned source | −.30                |
| GradB_norm   | gradient direction (normalised)    | 4/8   | Middle source      | −.12                |
| <b>GradB</b> | gradient direction (Eq. 4)         | 4/8   | Bottom source      | −.02                |

Table 2: **Task signatures evaluated against the measured transfer matrix of Table 1.** Deployment selects one source, so we score by best-source top-1 hit rate; robustness checks are in Appendix A.2. JointOT is our OTDD-style label-aware distributional baseline (Alvarez-Melis & Fusi, 2020; Nguyen et al., 2025a). Right: local intervention at the shared initialisation (three classification targets; protocol in Appendix A.2).

## 5 TRANSFER-SELECTIVE REPLAY

TSR is one instantiation of the §4 framework. The three conditions enter at two different levels: headroom admits no algorithm — it is a property of the benchmark, base, and budget, settled by the audit before any mechanism is designed (§4.1) — so the algorithm instantiates the remaining two, carrying transfer with persistent data (§4.2) and choosing sources by task signature (§4.3), while distillation takes over the stability objective that replay no longer serves (§5.3). We present the general form first (Algorithm 1), then the instantiation evaluated in this paper (Algorithm 2, Figure 4).

### 5.1 THE GENERAL ALGORITHM

**Algorithm 1** is what the framework requires of *any* instantiation: standard sequential training of an adapter, plus three replaceable modules — PROBE (represent the incoming task before training), ROUTE (weigh past tasks by predicted transfer), and REFERENCE (anchor each batch for stability) — with transfer carried by routed replay in every optimisation step. The modules are intentionally unspecified here; §5.2 fixes them.

**Notation.** Both algorithms share the following symbols.  $D_t$  is task  $t$ ’s training distribution, and  $b$  a minibatch;  $\theta$  is the trainable adapter,  $\theta_0$  the shared initialisation,  $\theta_t$  the snapshot frozen when task  $t$  ends. Each past task  $j$  leaves one record  $(g_j, \mathcal{M}_j, \theta_j)$  — signature, stored data subsample, snapshot — in the task memory  $\mathcal{P}$ ;  $w$  is the routing distribution over these records, and  $\theta^{\text{ref}}$  the frozen reference (teacher) of the stability term.  $\mathcal{L}_{\text{CE}}$  is the task loss of Eq. (1),  $\mathcal{L}_{\text{KD}}$  the stability regulariser (§5.3),  $\nabla_B$  the gradient w.r.t. the adapter’s zero-initialised low-rank factor  $B$  (Eq. 4), and  $f_\theta$  the model’s token-level predictive distribution. Hyperparameters: replay ratio  $\alpha$ , stability weight  $\lambda$ , routing temperature  $\tau$ , probe batches  $m$ , learning rate  $\eta$ .

The organising object is the *task memory*  $\mathcal{P} = \{(g_j, \mathcal{M}_j, \theta_j)\}_{j < t}$ : one record per past task, written once when the task ends and never updated. Its three fields answer an incoming task’s three needs — the signature says *whether*  $j$  should be drawn on, the data subsample is *what* gets replayed, and the era snapshot is *who* supervises that replay. Because the fields are bound in one record, a single routing decision returns coherent (data, teacher) pairs: replayed knowledge always arrives with the reference model that knew it best.

### 5.2 THE EVALUATED INSTANTIATION

**Algorithm 2** — the *routed triple* — fixes the three modules; the evaluated hyperparameters are listed with the experimental setup (§6.1). Figure 4 traces one task through the resulting pipeline; each choice carries its rationale from §4.

*Probe* — the mean probe-batch gradient at the fixed shared initialisation (Eq. 4; Figure 4a): the only tested signature that reliably finds oracle-best sources, label-aware where input-similarity signatures are not (§4.3); ten forward–backward passes, no training.

**Algorithm 1** General TSR

---

**Require:** task stream  $T_1, \dots, T_K$  with training data  $D_1, \dots, D_K$ ; initial parameters  $\theta_0$ ; modules PROBE, ROUTE, REFERENCE; hyperparameters  $\alpha, \lambda, \eta$

- 1:  $\mathcal{P} \leftarrow \{\}$ ;  $\theta \leftarrow \theta_0$
- 2: **for**  $t = 1, \dots, K$  **do**
- 3:    $g_t \leftarrow \text{PROBE}(D_t)$
- 4:    $w \leftarrow \text{ROUTE}(g_t, \{g_j\}_{j < t})$
- 5:   **for each** training step **do**
- 6:     with prob.  $1 - \alpha$ :  $b \sim D_t$ ;   otherwise:  $j \sim w, b \sim \mathcal{M}_j$
- 7:      $\theta^{\text{ref}} \leftarrow \text{REFERENCE}(b, \mathcal{P})$
- 8:      $\theta \leftarrow \theta - \eta \nabla_{\theta} [\mathcal{L}_{\text{CE}}(\theta; b) + \lambda \mathcal{L}_{\text{KD}}(\theta; \theta^{\text{ref}}, b)]$
- 9:   **end for**
- 10:    $\mathcal{M}_t \leftarrow$  subsample of  $D_t$ ;    $\theta_t \leftarrow \theta$ ;    $\mathcal{P} \leftarrow \mathcal{P} \cup \{(g_t, \mathcal{M}_t, \theta_t)\}$
- 11: **end for**

---

**Algorithm 2** Routed-triple TSR

---

**Require:** task stream  $T_1, \dots, T_K$  with training data  $D_1, \dots, D_K$ ; shared initialisation  $\theta_0$ ; hyperparameters  $m, \tau, \alpha, \lambda, \eta$

- 1:  $\mathcal{P} \leftarrow \{\}$ ;  $\theta \leftarrow \theta_0$
- 2: **for**  $t = 1, \dots, K$  **do**
- 3:    $g_t \leftarrow \frac{1}{m} \sum_{i=1}^m \nabla_B \mathcal{L}_{\text{CE}}(\theta_0; b_i), b_i \sim D_t$  ▷ PROBE
- 4:    $w_j \leftarrow \text{softmax}_{j < t}(\cos(g_j, g_t)/\tau)$  ▷ ROUTE
- 5:    $j^* \leftarrow \arg \max_{j < t} w_j$
- 6:   **for each** training step **do**
- 7:     draw  $u \sim \text{Uniform}(0, 1)$
- 8:     **if**  $u > \alpha$  **then**
- 9:       sample minibatch  $b \sim D_t$ ;    $\theta^{\text{ref}} \leftarrow \theta_{j^*}$  ▷ current-task batch
- 10:    **else**
- 11:      draw one source  $j \sim \text{Categorical}(w)$  ▷ past task  $j$  chosen with prob.  $w_j$
- 12:      sample minibatch  $b$  uniformly from  $\mathcal{M}_j$
- 13:       $\theta^{\text{ref}} \leftarrow \theta_j$  ▷ REFERENCE
- 14:    **end if**
- 15:     $\theta \leftarrow \theta - \eta \nabla_{\theta} [\mathcal{L}_{\text{CE}}(\theta; b) + \lambda \text{KL}(f_{\theta^{\text{ref}}}(b) \| f_{\theta}(b))]$
- 16:   **end for**
- 17:    $\mathcal{M}_t \leftarrow$  subsample of  $D_t$ ;    $\theta_t \leftarrow \theta$ ;    $\mathcal{P} \leftarrow \mathcal{P} \cup \{(g_t, \mathcal{M}_t, \theta_t)\}$
- 18: **end for**

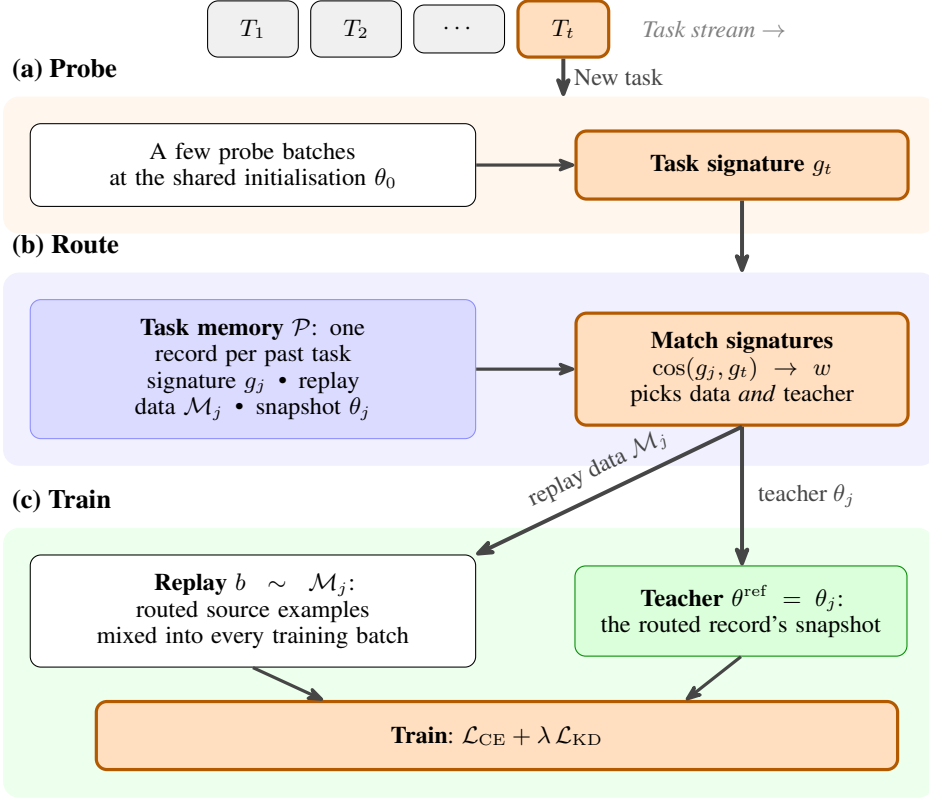
---

*Route* — softmax over signature cosines at temperature  $\tau$  (Figure 4b). The temperature makes the routing knob explicit:  $\tau \rightarrow 0$  recovers argmax selection,  $\tau \rightarrow \infty$  recovers uniform pooling (= ER), and the evaluated setting sits between the two extremes compared in Appendix B.3.

*Replay* — routed source data mixed into training at ratio  $\alpha$ , batch by batch (Figure 4c): data is the persistent carrier of §4.2, so the transferred signal re-enters at every optimisation step rather than only at initialisation.

*Reference* — each batch is distilled against the routed record’s era snapshot:  $\theta_j$  on batches replayed from  $\mathcal{M}_j$ , the top-routed  $\theta_{j^*}$  on current-task batches (Figure 4c). Why distillation owns stability — and why the teacher is an era snapshot — is the subject of §5.3.

Other instantiations of the three modules are configurations of the same skeleton (Appendix B.3).

Figure 4: **The Transfer-Selective Replay pipeline.**

### 5.3 KNOWLEDGE DISTILLATION FOR STABILITY

Once replay is reassigned to transfer, the stability term of the continual objective is unowned; TSR assigns it to distillation. Each training batch  $b$  is anchored by

$$\mathcal{L}_{\text{KD}}(\theta; b) = \text{KL}(f_{\theta^{\text{ref}}}(b) \parallel f_{\theta}(b)), \quad (5)$$

added to the task loss with weight  $\lambda$  (Algorithm 2): the KL is taken between the token-level predictive distributions of the frozen reference and the student on the same batch, and the reference switches with the batch’s origin —  $\theta_j$  on batches replayed from  $\mathcal{M}_j$ , the top-routed  $\theta_{j^*}$  on current-task batches.

The novelty is not the loss. It lies, first, in the *role assignment*: replay serves transfer, while distillation alone owns stability. It lies, second, in the *binding*: teachers are queryable era snapshots, routed jointly with the replay data through the task memory, so one routing decision returns both what to replay and whom to distil against, and the reference can answer on any batch, including current-task ones. To our knowledge, distillation as an explicitly routed stability module has no counterpart in recent continual learning with LLMs.

**What the teacher contributes.** A systematic teacher audit dissociates teacher *knowledge* from optimisation *anchoring*: teacher identity is second-order, the teacher’s value lives in its base-shared component, and every attempt to make distillation carry transfer reduces both transfer *and* stability. In this regime distillation functions as a stability regulariser — adjacent to reference-KL in RL (Teh et al., 2017) and to label-smoothing accounts of supervised distillation (Yuan et al., 2020; Mobahi et al., 2020), though the dissociation in continual LLM learning is, to our knowledge, new — and we assign it exclusively to stability. The one teacher property that consistently matters in long streams is *drift*: the most recent snapshot is increasingly biased toward the latest task, which is what motivates the era-matched reference. The full audit is reported in Appendix B.4.

## 6 EXPERIMENTS

Sections 4–5 measured *why* transfer should work and derived TSR as one instantiation. This section validates the framework under the standard continual-learning protocol, in three questions: does transfer improve low-budget continual learning (§6.2); what determines the size and shape of the gains (§6.3); and how much do the design choices matter (§6.4).

### 6.1 SETUP

**Models and streams.** Qwen2.5-0.5B/7B-Instruct with a single LoRA adapter (Hu et al., 2021) trained continually — the dominant regime for continual learning with LLMs. Two eight-task streams: TRACE-8 (Wang et al., 2023b) (heterogeneous: stance, finance, summarisation, code, science QA, arithmetic) in its canonical order, and NumGLUE-8 (Mishra et al., 2022) (one numerical-reasoning core in eight formats) in type order.

**Protocol.** We follow the standard sequential protocol of the systems we compare against (Wang et al., 2023a; Qian et al., 2025; Buzzega et al., 2020): one accumulating adapter trains through all eight tasks; after each task, all seen tasks are evaluated;  $n=5$  seeds per cell. We evaluate in the low-budget regime —  $N=50$  examples per task — where transfer is operationally interesting (§4.1); the protocol structure is the literature’s, the budget is the studied setting. All remaining configuration (adapter, optimiser, training budgets, and TSR’s module settings) is detailed in Appendix C.1, together with cost accounting and stream construction; order-robustness checks are in Appendix C.3.

**Metrics.** Training uses token-level cross-entropy. Evaluation reports exact-match accuracy on classification tasks and evaluation loss on generation tasks, where exact match is degenerate. Let  $A_{k,i}$  denote accuracy on  $T_i$ ’s test set after training on task  $T_k$ . We report the three standard sequential metrics (Lopez-Paz & Ranzato, 2017; Chaudhry et al., 2019a):

$$\text{Overall} = \frac{1}{K} \sum_{i=1}^K A_{K,i}, \quad (6)$$

$$\text{Plas} = \frac{1}{K} \sum_{i=1}^K A_{i,i}, \quad (7)$$

$$\text{BWT} = \frac{1}{K-1} \sum_{i=1}^{K-1} (A_{K,i} - A_{i,i}). \quad (8)$$

Overall is the final average accuracy; plasticity (Plas) is accuracy immediately after learning each task; and BWT is backward transfer, where 0 indicates perfect retention, positive values indicate improvement on earlier tasks, and negative values indicate forgetting.

**Baselines.** Naive sequential fine-tuning (SeqFT). *Rehearsal-free*: regularisation and distillation (EWC Kirkpatrick et al., 2017; LwF Li & Hoiem, 2017; SDFT Yang et al., 2024); parameter isolation (O-LoRA Wang et al., 2023a); and two NeurIPS’25 methods — GainLoRA (Liang et al., 2025), gated per-task LoRA branches with projection-constrained gates, and DEAL (Han et al., 2025), wavelet-filtered retention on a continually edited adapter (port details in Appendix C.1). *Rehearsal-based*: gradient constraints from memory samples (A-GEM Chaudhry et al., 2019a); DER++ Buzzega et al., 2020; and ER Chaudhry et al., 2019b — uniform replay over all past tasks — at a generous 50% replay ratio, with a 10% variant marking the budget spectrum. Zero-shot exact match is 0 under the strict answer format and is therefore omitted from Table 3.

### 6.2 DOES TRANSFER IMPROVE LOW-BUDGET CONTINUAL LEARNING?

We now test whether the transfer contribution of §3 improves continual learning under a fixed target-data budget (Table 3).

**Plasticity.** In the decomposition of §3, plasticity is where the transfer contribution should appear first, so we read Plas first. On TRACE, TSR posts the best plasticity at both scales (.439 at 0.5B, .670 at 7B): signature-weighted replay beats ER on the transfer axis, the controlled probe’s finding carried into the stream. On NumGLUE, methods that only chase the current task can edge Plas (SDFT .334

| Method            | TRACE-8 (0.5B)    |                   |                    | TRACE-8 (7B)      |                   |                    | NumGLUE-8 (0.5B)  |                   |                    | NumGLUE-8 (7B)    |                   |                    |
|-------------------|-------------------|-------------------|--------------------|-------------------|-------------------|--------------------|-------------------|-------------------|--------------------|-------------------|-------------------|--------------------|
|                   | Overall↑          | Plas↑             | BWT↑               | Overall↑          | Plas↑             | BWT↑               | Overall↑          | Plas↑             | BWT↑               | Overall↑          | Plas↑             | BWT↑               |
| SeqFT             | .264±.048         | .398±.015         | -.134±.058         | .584±.033         | .660±.001         | -.076±.031         | .284±.041         | .316±.018         | -.037±.053         | .555±.032         | <b>.587</b> ±.008 | -.037±.037         |
| EWC               | .245±.031         | .427±.004         | -.182±.031         | .599±.010         | .666±.011         | -.067±.014         | .279±.018         | .320±.014         | -.047±.030         | .554±.035         | .584±.009         | -.034±.034         |
| LwF               | .291±.045         | .424±.013         | -.133±.033         | .640±.024         | .668±.001         | -.029±.023         | .292±.014         | .313±.013         | -.024±.008         | .563±.015         | .580±.017         | -.020±.007         |
| SDFT              | .305±.070         | .427±.009         | -.122±.072         | .605±.011         | .664±.009         | -.059±.019         | .279±.033         | <b>.334</b> ±.016 | -.063±.027         | .553±.009         | .580±.011         | -.030±.011         |
| O-LoRA            | .233±.028         | .401±.007         | -.168±.030         | .617±.010         | .662±.010         | -.045±.005         | .281±.016         | .310±.028         | -.033±.014         | .567±.006         | .579±.014         | -.013±.010         |
| GainLoRA          | .374±.018         | .398±.022         | -.024±.005         | .654±.010         | .663±.009         | -.009±.005         | .297±.014         | .307±.017         | -.011±.004         | .558±.006         | .563±.005         | -.005±.004         |
| DEAL              | .372±.019         | .401±.003         | -.029±.022         | .224±.153         | .576±.078         | -.352±.088         | .253±.024         | .303±.013         | -.057±.023         | .418±.023         | .500±.006         | -.094±.023         |
| A-GEM             | .310±.019         | .418±.016         | -.107±.021         | .610±.032         | .662±.007         | -.052±.025         | .286±.031         | .325±.005         | -.045±.037         | .559±.025         | .586±.014         | -.031±.013         |
| DER++             | .220±.015         | .262±.022         | -.042±.037         | .264±.104         | .288±.096         | -.023±.081         | .210±.025         | .212±.028         | -.003±.007         | .393±.054         | .445±.010         | -.060±.051         |
| ER (10% replay)   | .406±.019         | .435±.008         | -.029±.025         | .638±.029         | .663±.005         | -.025±.026         | .321±.020         | .331±.013         | -.012±.011         | .563±.011         | .583±.013         | -.023±.016         |
| ER                | .451±.031         | .427±.024         | +.025±.020         | .656±.034         | .662±.018         | -.006±.046         | .336±.013         | .314±.011         | +.025±.010         | .578±.012         | .579±.015         | -.001±.019         |
| <b>TSR (ours)</b> | <b>.475</b> ±.030 | <b>.439</b> ±.017 | <b>+.035</b> ±.015 | <b>.684</b> ±.008 | <b>.670</b> ±.012 | <b>+.014</b> ±.011 | <b>.353</b> ±.016 | .330±.012         | <b>+.026</b> ±.007 | <b>.588</b> ±.017 | .582±.012         | <b>+.007</b> ±.022 |

Table 3: **Continual learning under the standard sequential protocol** (TRACE-8 / NumGLUE-8 at 0.5B / 7B). Mean  $\pm$  std over 5 seeds; bold = best per column. Overall and Plas are *absolute* accuracies; BWT is the change from post-task to end-of-stream accuracy; no row is normalised by the zero-shot reference (omitted: exact match is 0 under the strict answer format). Rules separate the no-mechanism baseline, the rehearsal-free family, the rehearsal-based family, and ours. Overall, Plas and BWT as in Eqs. (6)–(8); baselines and protocol in §6.1; routing-policy variants in Appendix B.3; per-task results in Appendix C.2.

at 0.5B, SeqFT .587 at 7B) — but they pay with the worst forgetting in their columns; plasticity bought by abandoning the past is not transfer.

**Overall accuracy.** Overall folds retention in, and TSR is best everywhere. The reading we care about is the budget one: with the same 50 target examples per task, selected replay reaches final performance that ER does not — a gain that would otherwise require additional target supervision, and that Figure 2(b) converts into equivalent target labels on the probe (§4.1). Stream-level margins are smaller than probe-level ones for a predicted reason: the stream itself gradually consumes pair-specific headroom — by the time a target arrives, its best source has often already been trained into the adapter.

**Stability.** The plasticity gains do not come at the expense of forgetting: TSR is the only method in Table 3 with *positive* BWT in all four settings. Replay incidentally protects the sources it draws on, and distillation anchors every batch to an undrifted era reference. **Under a fixed target budget, realised transfer buys higher plasticity and higher final accuracy at no retention cost.**

**Backbone robustness.** The same qualitative behaviour is observed on two additional backbone families (Llama-3.2-1B and Gemma-3-1B; Appendix C.3): replay-based methods remain strongest, and TSR again attains the best overall accuracy, plasticity, and backward transfer — the framework’s predictions are not specific to Qwen models.

### 6.3 WHAT DETERMINES TRANSFER?

The framework predicts that realised transfer depends jointly on transfer opportunity and task structure — and indeed the same method, protocol, and budget produce gains of very different size and shape across the four settings of Table 3. Our evaluation scope is deliberately **two-dimensional — target-data budget and source-task structure** — the same axes along which the reuse of previously collected data is evaluated in online RL (Ball et al., 2023). The differences follow the framework, not the method: two observations and two boundaries.

**Transfer follows task structure.** TRACE is heterogeneous: sources differ sharply and several hurt (Table 1), so *selection* is what pays — routing beats pooling on plasticity, and argmax selection is strongest on the pure-transfer probe (Appendix A.2). NumGLUE is the opposite regime — one arithmetic core in eight formats: headroom is large and dense (Appendix A.4.1), every source carries the core, and within such homogeneous families the framework predicts *pooling* rather than single-source replay, which the controlled probe confirms. The signature’s resolution also drops inside a family (its cosines compress; Appendix A.4.2) — a bound on the current instantiation, not on the framework, since the selection module is the replaceable one (§5.1). The routed triple stays best in the stream in both regimes because its soft routing spans them: sharp enough to select on TRACE, broad enough to keep coverage on NumGLUE.

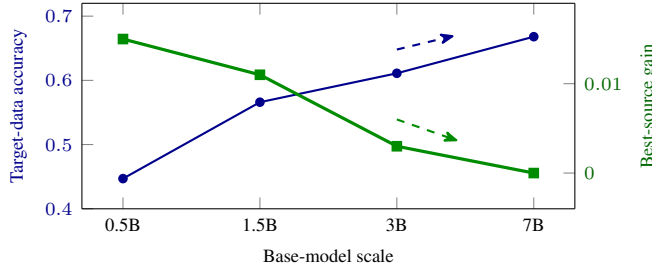


Figure 5: **Transfer follows headroom across model scale** (TRACE headroom audit per Qwen2.5 scale; error bands omitted for clarity).

**Transfer follows headroom.** Figure 5 repeats the headroom audit across Qwen2.5 scales on TRACE: as the base grows, the target-data arm saturates on its own and the best-source gain decays monotonically to zero; at 7B the oracle buys nothing. This is a probe-level statement about opportunity, not a method comparison, and it agrees with Table 3: every TRACE Plas column flattens at 7B — no policy can transfer where no opportunity remains — while Overall still separates through retention.

NumGLUE shows the graded version of the same law: its in-domain headroom persists at 7B, but the fraction other tasks can supply compresses ( $+0.033 \rightarrow +0.003$ ) — scale erodes what tasks share before it erodes what each still individually lacks. Notably, signature picks are scale-stable (7B agrees with 0.5B on 7/8 targets), so signatures can be computed once on a small proxy model.

**Boundaries.** The framework predicts its own two edges, and both are observed: *saturation* — on benchmarks without headroom, transfer is unmeasurable by any mechanism; and *generic transfer* — the regime where most sources provide similar benefit, so source identity matters little: on six Super-NaturalInstructions pairs headroom exists and source data helps, but sources are near-interchangeable and the value of selection is small (Appendix A.3). **Thus, task structure determines how transfer should be realised, whereas headroom determines whether it can be realised at all.**

#### 6.4 DESIGN CHOICES

The framework specifies *what* must be decided, not *how*: Algorithm 2 leaves implementation choices open, and we ablate these — how replay is routed, and whether task order still matters once routing is available.

**Routing.** The routing module decides whether replay comes from a single selected source or a weighted combination; the framework is agnostic, so we compare argmax selection, soft routing, and uniform pooling. In the stream, soft routing — the routed triple — beats both extremes and its single-component variants (Table 3; Appendix B.3); on the controlled probe, even a softmax-weighted top-3 mixture *lowers* mean accuracy against the single selected source. The reason is the shape of the matrix: on heterogeneous streams it is sparse — the second- and third-best sources are often weak or harmful, and averaging them dilutes transfer. **Sparse transfer matrices favour selection over averaging.**

**Task order.** If the matrix is usable prospectively, ordering tasks so strong sources precede their targets should help. We compare the canonical order, the greedy best-source curriculum (fixed before evaluation), its reversal, and arbitrary reshuffles, at both scales (Appendix C.3). The replay family moves by at most .03 under any of them, on both streams and both scales: replay already delivers the relevant source at training time, so a curriculum has little left to contribute. **Selection replaces curriculum.**

## 7 DISCUSSION

### 7.1 IMPLICATIONS

This work does not propose another replay algorithm; it changes the questions asked before one is designed. Three reversals follow. *Replay*: long treated as a forgetting-mitigation tool, replay is the cheapest persistent carrier of transfer — pointing it at the new task turns a stability mechanism into a plasticity mechanism, and distillation absorbs the stability role it leaves behind. *Transfer*: what looked like a parameter-reuse problem is a data-selection problem — the transferred signal must survive optimisation, and among persistent carriers data is the cheapest — so the productive question is not how to merge parameters but which examples to select. *Benchmarks*: a continual-learning benchmark measures more than accuracy — its headroom and transfer structure decide whether transfer is measurable on it at all, and both belong in its specification, alongside whether the task order is canonical, arbitrary, or optimised; selection makes replay-based learners order-insensitive while order dominates learners without data access (§6.4), so curricula and selection are substitute uses of the same measured matrix.

### 7.2 LIMITATIONS AND OPEN QUESTIONS

The framework was validated in the low-budget regime, where transfer opportunity is measurable; whether it yields equally useful mechanisms under richer supervision is an empirical question. Within this regime, each limitation pairs with the question it opens.

*Storage*. Replay assumes access to past task data. The buffer is small ( $\sim 100$  examples per task,  $\sim 3$  MB for 15 tasks, favourable versus per-task adapter libraries), but privacy-constrained deployments cannot store raw examples — opening the question of whether transfer can be *carried differently*: synthetic or distilled replay and retrieval are candidates, judged by the same persistence criterion (§4.2).

*The current instantiation*. The implementation instantiates the framework with one task signature and one replay policy; neither is claimed optimal. Signature resolution drops within homogeneous families (§6.3) — opening the question of whether transfer can be *predicted better*: task signatures that retain within-family resolution, and signatures for modalities beyond text.

*Evaluation*. Existing benchmarks are designed to expose forgetting, not transfer — headroom had to be audited here before any mechanism could be tested — opening the question of whether transfer can be *benchmarked directly*: streams with reported headroom, a published pairwise matrix, and known negative cells, so that transfer is evaluated as directly as forgetting is today.

Beyond language, the conditions are stated at the level of tasks and data: vision and multimodal streams, and budget-allocation problems outside supervised language modelling — e.g., allocating simulation budget for scientific surrogates (Meng et al., 2026) — pose the same fixed-budget transfer question.

### 7.3 CONCLUSION

*Transfer in continual learning is a data-selection problem rather than a parameter-reuse problem.* The two carrier studies point at one law: continual optimisation preserves persistent signals and suppresses transient, task-specific ones. TSR is one instantiation; the framework is intended to outlive it. The particular signature, replay policy, and stability mechanism are all replaceable — what we hope remains is the discipline: before proposing a transfer mechanism, first ask whether transfer is possible, what can carry it, and whom it should come from. We hope continual learning comes to evaluate transfer as systematically as it has evaluated forgetting.

## 8 ACKNOWLEDGMENTS

We gratefully acknowledge the support of NSF IIS-2313131, NSF IIS-2332475 and NSF IIS-2543755, the NSF-Simons AI-Institute for the Sky (SkAI) via grants NSF AST-2421845 and Simons Foundation MPS-AI00010513, and the University of Chicago’s Research Computing Center for their support of this work.

## REFERENCES

- Alessandro Achille, Michael Lam, Rahul Tewari, Avinash Ravichandran, Subhansu Maji, Charles C Fowlkes, Stefano Soatto, and Pietro Perona. Task2vec: Task embedding for meta-learning. In *ICCV*, 2019.
- Mubashara Akhtar, Anka Reuel, Prajna Soni, et al. When AI benchmarks plateau: A systematic study of benchmark saturation. *arXiv preprint arXiv:2602.16763*, 2026.
- Rahaf Aljundi, Eugene Belilovsky, Tinne Tuytelaars, Laurent Charlin, Massimo Caccia, Min Lin, and Lucas Page-Caccia. Online continual learning with maximal interfered retrieval. In *Advances in Neural Information Processing Systems*, 2019a.
- Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning. In *Advances in Neural Information Processing Systems*, 2019b.
- David Alvarez-Melis and Nicolo Fusi. Geometric dataset distances via optimal transport. In *Advances in Neural Information Processing Systems*, 2020.
- Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, et al. Continual learning bench: Evaluating frontier AI systems in real-world stateful environments. *arXiv preprint arXiv:2606.05661*, 2026.
- Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, 2023.
- Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems*, 2020.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations*, 2019a.
- Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K Dokania, Philip HS Torr, and Marc’Aurelio Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019b.
- Yan Chen, Taojie Zhu, Meng Zhang, Xin Chen, Jiaqi Huang, Dongyang Xu, and Yizhi Wang. On-policy replay for continual supervised fine-tuning. *arXiv preprint arXiv:2605.29495*, 2026.
- Minsik Choi and Geewook Kim. Decentralized instruction tuning: Conflict-aware splitting and weight merging. In *International Conference on Machine Learning*, 2026.
- Alexandra Chronopoulou, Matthew E Peters, Alexander Fraser, and Jesse Dodge. Adaptersoup: Weight averaging to improve generalization of pretrained language models. In *Findings of EACL*, 2023.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- Chris Fifty, Ehsan Amid, Zhe Zhao, Tianhe Yu, Rohan Anil, and Chelsea Finn. Efficiently identifying task groupings for multi-task learning. In *Advances in Neural Information Processing Systems*, 2021.
- Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Xiao Han, Zimo Zhao, Wanyu Wang, Maolin Wang, Zitao Liu, Yi Chang, and Xiangyu Zhao. Data efficient adaptation in large language models via continuous low-rank fine-tuning. In *Advances in Neural Information Processing Systems*, 2025.
- Danny Hernandez, Jared Kaplan, Tom Henighan, and Sam McCandlish. Scaling laws for transfer. *arXiv preprint arXiv:2102.01293*, 2021.

- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuezhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. Lorahub: Efficient cross-task generalization via dynamic LoRA composition. *arXiv preprint arXiv:2307.13269*, 2023.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.
- Khurram Javed and Martha White. Meta-learning representations for continual learning. In *Advances in Neural Information Processing Systems*, 2019.
- Zixuan Ke and Bing Liu. Continual learning of natural language processing tasks: A survey. *arXiv preprint arXiv:2211.12701*, 2022.
- Zixuan Ke, Bing Liu, and Xingchang Huang. Continual learning of a mixed sequence of similar and dissimilar tasks. In *Advances in Neural Information Processing Systems*, 2020.
- Zixuan Ke, Bing Liu, Nianzu Ma, Hu Xu, and Lei Shu. Achieving forgetting prevention and knowledge transfer in continual learning. In *Advances in Neural Information Processing Systems*, 2021a.
- Zixuan Ke, Hu Xu, and Bing Liu. Adapting BERT for continual learning of a sequence of aspect sentiment classification tasks. In *NAACL*, 2021b.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences*, 2017.
- Fanshuang Kong, Richong Zhang, Zhijie Nie, et al. MOMA: Masked orthogonal matrix alignment for zero-additional-parameter model merging. *arXiv preprint arXiv:2412.13526*, 2024.
- Shambhavi Krishna, Atharva Naik, Chaitali Agarwal, Sudharshan Govindan, Taesung Lee, and Haw-Shiuan Chang. Latent traits and cross-task transfer: Deconstructing dataset interactions in LLM fine-tuning. *arXiv preprint arXiv:2509.13624*, 2025.
- Jiachun Li, Yang Meng, David Simchi-Levi, and Chonghuan Wang. Personalized incentive alignment: Correcting utility-driven selection bias in a/b tests. In *Proceedings of the 29th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2026.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.
- Yan-Shuo Liang, Jia-Rui Chen, and Wu-Jun Li. Gated integration of low-rank adaptation for continual learning of large language models. In *Advances in Neural Information Processing Systems*, 2025.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017.
- Yang Meng, Ruoxi Jiang, Zhuokai Zhao, Chong Liu, Rebecca Willett, and Yuxin Chen. Accelerating PDE surrogates via RL-guided mesh optimization. In *The 29th International Conference on Artificial Intelligence and Statistics*, 2026.
- Swaroop Mishra, Arindam Mitra, Neeraj Varshney, Bhavdeep Sachdeva, Peter Clark, Chitta Baral, and Ashwin Kalyan. Numglue: A suite of fundamental yet challenging mathematical reasoning tasks. In *Proceedings of ACL*, 2022.
- Hossein Mobahi, Mehrdad Farajtabar, and Peter Bartlett. Self-distillation amplifies regularization in Hilbert space. In *Advances in Neural Information Processing Systems*, 2020.

- Saleh Momeni and Bing Liu. Achieving upper bound accuracy of joint training in continual learning. *arXiv preprint arXiv:2502.12388*, 2025.
- Saleh Momeni, Sahisnu Mazumder, Zixuan Ke, and Bing Liu. In-context continual learning assisted by an external continual learner. In *COLING*, 2025.
- Khai Nguyen, Hai Nguyen, Tuan Pham, and Nhat Ho. Lightspeed geometric dataset distance via sliced optimal transport. *arXiv preprint arXiv:2501.18901*, 2025a.
- Thinh Nguyen, Cuong N Nguyen, Quang Pham, Binh T Nguyen, Savitha Ramasamy, Xiaoli Li, and Cuong V Nguyen. Sequence transferability and task order selection in continual learning. *arXiv preprint arXiv:2502.06544*, 2025b.
- Oleksiy Ostapenko, Zhan Su, Edoardo Maria Ponti, Laurent Charlin, Nicolas Le Roux, Matheus Pereira, Lucas Caccia, and Alessandro Sordoni. Towards modular LLMs by building and reusing a library of LoRAs. *arXiv preprint arXiv:2405.11157*, 2024.
- Jason Phang, Thibault Févry, and Samuel R Bowman. Sentence encoders on STILTs: Supplementary training on intermediate labeled-data tasks. *arXiv preprint arXiv:1811.01088*, 2018.
- Yu-Yang Qian, Yuan-Ze Xu, Zhen-Yu Zhang, Peng Zhao, and Zhi-Hua Zhou. TreeLoRA: Efficient continual learning via layer-wise LoRAs guided by a hierarchical gradient-similarity tree. In *ICML*, 2025.
- Fuli Qiao and Mehrdad Mahdavi. Merge before forget: A single LoRA continual learning via continual merging. *arXiv preprint arXiv:2512.23017*, 2025.
- Chengwei Qin and Shafiq Joty. LFPT5: A unified framework for lifelong few-shot language learning based on prompt tuning of T5. In *International Conference on Learning Representations*, 2022.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Madian Khabza, Mike Lewis, and Amjad Almahairi. Progressive prompts: Continual learning for language models. In *International Conference on Learning Representations*, 2023.
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. iCaRL: Incremental classifier and representation learning. In *CVPR*, 2017.
- Matthew Riemer, Ignacio Cases, Robert Ajemian, Miao Liu, Irina Rish, Yuhai Tu, and Gerald Tesaro. Learning to learn without forgetting by maximizing transfer and minimizing interference. In *International Conference on Learning Representations*, 2019.
- Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Ibne Farabi Shihab, Fariya Afrin, and Anuj Sharma. Dynamic proxy-mixing: Transferring replay controllers from small to large models for continual instruction tuning. *arXiv preprint arXiv:2606.00400*, 2026.
- James Seale Smith et al. Adaptive memory replay for continual learning. In *CVPR Workshops*, 2024. *arXiv:2404.12526*.
- Huashan Sun, Yizhe Yang, Yinghao Li, Jiawei Li, and Yang Gao. Unveiling and addressing pseudo forgetting in large language models. In *Findings of ACL*, 2025.
- Pavan Kalyan Tankala, Shubhra Mishra, Satya Lokam, and Navin Goyal. CurLL: A developmental framework to evaluate continual learning in language models. In *BabyLM Workshop*, 2025.
- Matthew E Taylor and Peter Stone. Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research*, 10:1633–1685, 2009.

- Yee Whye Teh, Victor Bapst, Wojciech M Czarnecki, John Quan, James Kirkpatrick, Raia Hadsell, Nicolas Heess, and Razvan Pascanu. Distral: Robust multitask reinforcement learning. In *Advances in Neural Information Processing Systems*, 2017.
- Tom Veniat, Ludovic Denoyer, and Marc’ Aurelio Ranzato. Efficient continual learning with modular networks and task-driven priors. In *International Conference on Learning Representations*, 2021.
- Tu Vu, Tong Wang, Tsendsuren Munkhdalai, Alessandro Sordoni, Adam Trischler, Andrew Mattarella-Micke, Subhansu Maji, and Mohit Iyyer. Exploring and predicting transferability across NLP tasks. In *EMNLP*, 2020.
- Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024a.
- Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. Orthogonal subspace learning for language model continual learning. In *Findings of EMNLP*, 2023a.
- Xiao Wang, Yuansen Zhang, Tianze Chen, Songyang Gao, Senjie Jin, Xianjun Yang, Zhiheng Xi, Rui Zheng, Yicheng Zou, Tao Gui, et al. TRACE: A comprehensive benchmark for continual learning in large language models. *arXiv preprint arXiv:2310.06762*, 2023b.
- Yifan Wang et al. InsCL: A data-efficient continual learning paradigm for fine-tuning large language models with instructions. In *NAACL*, 2024b. arXiv:2403.11435.
- Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning. In *International Conference on Machine Learning*, 2024.
- Shuo Yang, Kun-Peng Ning, Yu-Yang Liu, et al. Is parameter collision hindering continual learning in LLMs? In *COLING*, 2025.
- Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. In *ACL*, 2024.
- Li Yuan, Francis EH Tay, Guilin Li, Tao Wang, and Jiashi Feng. Revisiting knowledge distillation via label smoothing regularization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- Amir R Zamir, Alexander Sax, William Shen, Leonidas J Guibas, Jitendra Malik, and Silvio Savarese. Taskonomy: Disentangling task transfer learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Xin Zhang, Liang Bai, Xian Yang, and Jiye Liang. C-LoRA: Continual low-rank adaptation for pre-trained models. *arXiv preprint arXiv:2502.17920*, 2025.
- Zhuokai Zhao, Yibo Jiang, and Yuxin Chen. Direct acquisition optimization for low-budget active learning. *arXiv preprint arXiv:2402.06045*, 2024.
- Junhao Zheng, Xidi Cai, Shengjie Qiu, and Qianli Ma. Spurious forgetting in continual learning of language models. In *International Conference on Learning Representations*, 2025. arXiv:2501.13453.

## APPENDIX

**Appendix guide.** Appendix A collects the benchmark-level evidence for the framework of §4: the four regimes — PP15 (no headroom), TRACE (controlled transfer measurement), SuperNI (generic transfer), and NumGLUE (homogeneous transfer). Appendix B collects the implementation evidence behind §5: carrier, replay, routing, and distillation. Appendix C is the experimental record behind §6: setup, per-task results, robustness, and reproducibility. Unless stated otherwise, appendix results follow the probe protocol of §4.1 and the stream protocol of §6.1.

## A BENCHMARK REGIMES

## A.1 PP15: THE ZERO-HEADROOM REGIME

Under the audit of §4.1, PP15 with an instruction-tuned base leaves essentially no transfer opportunity: ten times more target supervision improves accuracy by only +0.02 on average. Consistently, the best of fourteen parameter-reuse configurations (oracle-selected source; initialisation, perturbation, moment, and projection families;  $n=5$ ) improves the no-transfer baseline by +0.005 — within one seed standard deviation — and the mean across configurations is  $-0.008$ . No transient advantage survives past  $\sim 50$  steps under early-budget checks.

**Protocol caution.** An earlier oracle sweep under an asymmetric merge protocol reported peer-pair gains up to +0.96; controlled re-measurement attributed  $\sim 97\%$  of that signal to an eval-path artefact of the merge rule — oracle definitions must be protocol-clean.

## A.2 TRACE-8: CONTROLLED TRANSFER MEASUREMENT

This appendix records the controlled protocol behind the transfer claims of §4: the five probe arms (target-data, target-full, mixed with a chosen source, random and hindsight-oracle selection) run per target on a fresh adapter.

**Budget sweep.** The sweep behind Figure 2 runs every arm at  $N \in \{10, 20, 50, 100, 200, 500\}$  on the five TRACE classification targets (sources always contribute 500 examples; mixing 50/50). Three regularities hold across targets and confirm the flagship reading: the opportunity shrinks monotonically as  $N$  grows; the selected-replay gain tracks it and concentrates on matrix-strong targets — numglue-cm captures 35% of its remaining opportunity at  $N=10$  (+.066, a  $2\times$  label efficiency) — while matrix-weak targets sit near zero and pay for the forced mixture once saturated; and the parameter-init arm never separates from zero at any budget.

**Task-signature robustness.** Each signature uses 10 probe batches and is scored by best-source top-1 hit rate against the measured matrix (chance  $\approx 14\%$ , i.e.  $1/7$ ). Two observations complement Table 2. *The signal is distributed across depth:* restricting the gradient signature to the last four blocks (`GradB_last4`) drops top-1 from  $4/8$  to  $3/8$ , so no single depth carries the alignment. *The fixed shared initialisation matters:* every task’s gradient is taken at the same reference point, so the signatures form a stable, training-free pairwise object — unlike gradient similarities computed at evolving weights during CL (e.g., for adapter routing (Qian et al., 2025)).

**Local intervention.** *Protocol.* Three classification targets spanning the structure of Table 1 (C-STANCE: weak, mixed structure; ScienceQA: negative sources present; NumGLUE-ds: strong positive transfer). Each repetition draws three independent batches — one for the target gradient, one for a source gradient, one held-out target batch for evaluation — and applies pure parameter perturbations (no optimiser state), all normalised to the same norm  $\epsilon$ : the *source-only* effect is  $L_t(\theta - \epsilon \hat{g}_s) - L_t(\theta)$ , and the *marginal* effect adds the source step after one equal-norm target step  $\theta_T$ , i.e.  $L_t(\theta_T - \epsilon \hat{g}_s) - L_t(\theta_T)$ ; a second, independent target batch added to  $\theta_T$  gives the equal-compute reference. To first order the source-only effect is  $-\epsilon \nabla L_t(\theta)^\top \hat{g}_s$ , so alignment should control immediate descent. We use  $\epsilon \in \{0.25, 0.5, 1\} \times$  the measured one-step parameter displacement and 30 repetitions per state. *Results.* At the shared initialisation, the within-target rank correlation between gradient cosine and immediate loss reduction is  $\rho = .93$  (source-only) and  $.87$  (marginal)

over 90 repetitions. The group ordering of Table 2 is strict at all three scales, with helpful-step rates of 100%/96%/67% for top/middle/bottom sources and the bottom pair turning harmful at the largest scale. The stored signature preserves the current gradient geometry (Spearman .85), so the quantity TSR routes on identifies the sources whose gradients provide compatible descent; the top-aligned pair recovers roughly 28% of the descent an additional target batch would provide, without additional target labels. *Scope*. The intervention is diagnostic at the shared initialisation, where TSR measures its signatures. Later checkpoints were also examined; the one-step probe loses power once the 50-example target budget is largely fitted, including for target gradients themselves. We therefore scope the claim to the shared initialisation; long-horizon effects are what the stream experiments measure.

### A.3 SUPERNI: THE GENERIC-TRANSFER REGIME

SuperNI illustrates the generic-transfer boundary predicted by the framework (§6.3). On six held-out Super-NaturalInstructions pairs ( $n=5$ ), headroom exists: additional in-domain supervision improves loss on 6/6 pairs — PP15’s zero-headroom regime, not TRACE’s, is the anomaly. Source data helps: mixed training improves target loss on 5/6 pairs. But selection has almost no value: related and unrelated sources help nearly equally (+0.04 loss difference). The selection premium collapses exactly where source-quality variance does; we report this boundary rather than average over it.

### A.4 NUMGLUE-8: THE HOMOGENEOUS REGIME

#### A.4.1 HEADROOM AUDIT

NumGLUE-8 casts the eight task types of the NumGLUE suite (Mishra et al., 2022) as the homogeneous regime: one numerical-reasoning core in eight formats (types 4/5 are TRACE’s numglue-cm/ds). The §4.1 audit gives three regularities. Headroom exists and persists across scale: 7/8 types retain accuracy headroom at 0.5B and at 7B ( $n=5$ ). Pooling beats single-source selection on the probe at both scales (0.348 vs. 0.342 at 0.5B, 0.600 vs. 0.585 at 7B) — the pooling prediction of §4.3. And the within-family oracle ceiling compresses from +0.033 to +0.003 across the same scales: in-domain headroom persists while the share other tasks can supply shrinks.

#### A.4.2 PAIRWISE TRANSFER MATRIX

Full transfer matrix ( $\Delta\text{acc}$ ) behind §6.3 ( $\Delta\text{acc}$  of 50/50 joint training vs. target-data;  $n=5$ ; green = helps, red = hurts, bold = column best; T4/T5 are TRACE’s numglue-cm/ds).

| src\tgt | T1            | T2            | T3            | T4            | T5            | T6            | T7            | T8            |
|---------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| T1      | —             | −0.013        | −0.015        | +0.015        | +0.008        | −0.015        | −0.018        | +0.006        |
| T2      | <b>+0.060</b> | —             | −0.015        | +0.041        | +0.016        | −0.011        | +0.004        | +0.017        |
| T3      | +0.012        | +0.006        | —             | +0.013        | −0.003        | −0.002        | <b>+0.005</b> | −0.006        |
| T4      | <b>+0.054</b> | +0.001        | −0.003        | —             | <b>+0.041</b> | −0.013        | −0.015        | <b>+0.047</b> |
| T5      | +0.012        | −0.007        | −0.027        | +0.037        | —             | −0.024        | −0.026        | −0.009        |
| T6      | +0.024        | <b>+0.015</b> | −0.006        | +0.000        | +0.007        | —             | −0.005        | +0.009        |
| T7      | <b>+0.048</b> | −0.012        | <b>+0.018</b> | <b>+0.041</b> | +0.011        | −0.007        | —             | −0.009        |
| T8      | <b>+0.054</b> | −0.009        | −0.030        | <b>+0.048</b> | +0.009        | <b>+0.005</b> | −0.017        | —             |

Table 4: Pairwise transfer matrix on NumGLUE-8.

## B IMPLEMENTATION EVIDENCE

### B.1 THE CARRIER

Our evidence rules out parameter reuse *as transient optimiser state*: initialising from, perturbing toward, or injecting the moments of a past adapter into a module that then continues training — the form parameter reuse takes in dominant LLM-CL practice. It does *not* contradict the two families of parameter-side successes in the literature, and in fact explains both. Inference-time composition is never trained against — nothing erases it. And architectural sharing in the Progressive-Networks /

CAT / CTR lineage keeps past parameters *frozen and present at every forward pass* — a persistent signal, like data, rather than a one-shot initial condition. The unifying property is persistence: *signals that participate in every training step survive; one-shot parameter states are washed out*. The property grades, rather than merely splits, the mechanism space: an L2 anchor toward the source adapter — a *persistent parameter* signal — captures roughly two-thirds of data mixing’s gain at the better anchor strength where one-shot initialisation captures none (mean 0.4526 vs. 0.4472 target-data and 0.4547 data; Appendix B.2). The three carriers therefore occupy the persistence axis exactly as predicted.

**A structural intuition.** LoRA’s update  $BA$  is a *residual* on the shared base: it encodes what remains of a task after the base has accounted for what the tasks share. Two related tasks can have near-orthogonal residuals (the shared part was subtracted), which is why parameter similarity under-determines transfer. What related tasks actually share — the capability the base lacks — lives in their data, and is re-suppliable at every step.

## B.2 REPLAY AND ANCHOR ABLATIONS

**Persistent parameter anchoring (L2-toward-source).** Training the target with  $L = \text{CE} + \mu \|\theta - \theta_{\text{source}}\|^2$  keeps the source adapter present at every step — a persistent signal whose carrier is *parameters*. On the five oracle pairs ( $n=5$ ): mean accuracy 0.4526 at  $\mu=0.1$  and 0.4410 at  $\mu=1.0$ , vs. 0.4472 target-data and 0.4547 for data mixing. Persistence is necessary (the one-shot init captures nothing) but not sufficient: the data carrier remains stronger at every  $\mu$  we probed.

## B.3 ROUTING-POLICY VARIANTS

The routed triple has two knobs: the routing temperature  $\tau$  (argmax at  $\tau \rightarrow 0$ , uniform pooling at  $\tau \rightarrow \infty$ ) and the teacher binding (recent snapshot vs. the routed batch’s era snapshot). All single-component variants, same protocol and budgets as Table 3:

| Method                                                    | TRACE-8 (0.5B)                  |                                 |                                  | TRACE-8 (7B)                    |                                 |                                  | NumGLUE-8 (0.5B)                |                                 |                                  | NumGLUE-8 (7B)                  |                                 |                                  |
|-----------------------------------------------------------|---------------------------------|---------------------------------|----------------------------------|---------------------------------|---------------------------------|----------------------------------|---------------------------------|---------------------------------|----------------------------------|---------------------------------|---------------------------------|----------------------------------|
|                                                           | Overall                         | Plas                            | BWT                              | Overall                         | Plas                            | BWT                              | Overall                         | Plas                            | BWT                              | Overall                         | Plas                            | BWT                              |
| ER (uniform pooling; $\tau \rightarrow \infty$ )          | .451 $\pm$ .031                 | .427 $\pm$ .024                 | +.025 $\pm$ .020                 | .656 $\pm$ .034                 | .662 $\pm$ .018                 | −.006 $\pm$ .040                 | .336 $\pm$ .013                 | .314 $\pm$ .011                 | +.025 $\pm$ .010                 | .578 $\pm$ .012                 | .579 $\pm$ .015                 | −.001 $\pm$ .019                 |
| argmax selection ( $\tau \rightarrow 0$ , recent teacher) | .439 $\pm$ .033                 | .430 $\pm$ .014                 | +.010 $\pm$ .019                 | .675 $\pm$ .010                 | .668 $\pm$ .015                 | +.007 $\pm$ .009                 | .308 $\pm$ .013                 | .308 $\pm$ .012                 | +.000 $\pm$ .016                 | .572 $\pm$ .011                 | .577 $\pm$ .011                 | −.005 $\pm$ .007                 |
| soft routing (recent teacher)                             | .459 $\pm$ .011                 | .436 $\pm$ .015                 | +.023 $\pm$ .006                 | .664 $\pm$ .026                 | .668 $\pm$ .010                 | −.004 $\pm$ .028                 | .338 $\pm$ .010                 | .316 $\pm$ .011                 | +.025 $\pm$ .018                 | .572 $\pm$ .014                 | .577 $\pm$ .012                 | −.005 $\pm$ .026                 |
| pool-hybrid (recent teacher)                              | .459 $\pm$ .020                 | .435 $\pm$ .007                 | +.024 $\pm$ .014                 | .669 $\pm$ .014                 | .670 $\pm$ .017                 | −.001 $\pm$ .019                 | .333 $\pm$ .009                 | .318 $\pm$ .014                 | +.018 $\pm$ .015                 | .566 $\pm$ .040                 | .575 $\pm$ .016                 | −.010 $\pm$ .030                 |
| <b>routed triple (full TSR)</b>                           | <b>.475<math>\pm</math>.030</b> | <b>.439<math>\pm</math>.017</b> | <b>+.035<math>\pm</math>.015</b> | <b>.684<math>\pm</math>.008</b> | <b>.670<math>\pm</math>.012</b> | <b>+.014<math>\pm</math>.011</b> | <b>.353<math>\pm</math>.016</b> | <b>.330<math>\pm</math>.012</b> | <b>+.026<math>\pm</math>.007</b> | <b>.588<math>\pm</math>.017</b> | <b>.582<math>\pm</math>.012</b> | <b>+.007<math>\pm</math>.022</b> |

Table 5: Routing-policy variants of TSR under the standard stream protocol.

Each ablated variant loses on the axis its missing component serves (argmax loses coverage, pooling loses selection, recent teachers lose the undrifted reference).

## B.4 DISTILLATION: THE FULL ABLATION RECORD

This appendix records the distillation audit referenced in §6.3. All experiments: PP15, sequential CL with a single shared LoRA,  $q, k, v, o$  projections, 500 steps/task,  $n=5$  seeds, final mean accuracy (AVG) unless noted. The cumulative conclusion: KD in this setting is a trajectory regulariser anchored to the shared base — a stability mechanism — and cannot be converted into a transfer channel by teacher choice, loss reshaping, timing, weighting, or snapshot source.

### B.4.1 TEACHER IDENTITY IS IRRELEVANT (ROUTING CEILING)

If KD transferred teacher-specific knowledge, teacher choice should matter. It does not:

Even the per-batch oracle — the tightest upper bound on routing benefit — adds nothing over a random teacher. The gain over no distillation (+0.13) is therefore attributable to having *an* anchor, not to choosing the right one.

### B.4.2 SIX LOSS-RESHAPING INTERVENTIONS ALL FAIL

Each intervention was designed to amplify or isolate teacher-specific signal; every one hurts:

Removing the base-shared component or repelling the student from the base is catastrophic — direct evidence that the value of KD lies in the base-shared anchor, not in task-specific teacher content.

| Teacher selection                 | AVG               | note                           |
|-----------------------------------|-------------------|--------------------------------|
| cosine prototype (default)        | $0.724 \pm 0.018$ |                                |
| random teacher per batch          | $0.715 \pm 0.022$ | ties cosine                    |
| per-batch oracle (min-CE teacher) | $0.715 \pm 0.004$ | <i>ceiling</i> ; ties random   |
| latest snapshot only              | $0.703 \pm 0.001$ | systematically biased, $-0.02$ |
| anchor to frozen base             | $0.659 \pm 0.008$ | no snapshot at all             |
| no distillation (sequential FT)   | $0.584 \pm 0.022$ |                                |

Table 6: Teacher-identity ablation for the distillation audit.

| Variant                                           | AVG               | vs. plain KD (0.724) |
|---------------------------------------------------|-------------------|----------------------|
| amplify teacher LoRA scale ( $\gamma=4$ )         | 0.493             | $-0.231$             |
| distill on residual (teacher $-$ base)            | 0.516             | $-0.208$             |
| push student away from base ( $\lambda=10^{-3}$ ) | 0.234             | $-0.490$             |
| selective KL (gate by teacher correctness)        | 0.602             | $-0.122$             |
| mixup CE                                          | collapse          | —                    |
| struggle-gate / margin transfer / hidden-MSE      | 0.638/0.600/0.681 | all negative         |

Table 7: Loss-reshaping interventions on the distillation term.

#### B.4.3 TIMING AND WEIGHT: REGULARISER SIGNATURES

KD interleaved with CE beats KD applied as a post-hoc phase (endpoint correction), across two initialisation modes:

| Init       | KD during CE      | KD after CE       |
|------------|-------------------|-------------------|
| QR         | $0.673 \pm 0.015$ | $0.651 \pm 0.010$ |
| routed mix | $0.627 \pm 0.016$ | $0.614 \pm 0.023$ |

Table 8: Distillation timing: interleaved versus post-hoc.

The weight curve is shallow over an order of magnitude (AVG 0.673–0.683 for  $\lambda \in [0.1, 2]$ ) — no narrow optimum, no collapse at either end: the canonical signature of a soft regulariser.

#### B.4.4 SNAPSHOT SOURCE IS IRRELEVANT

Anchoring to an accumulated merged snapshot versus a routed per-task snapshot makes no measurable difference (QR init: 0.673 vs. 0.673; routed-mix init: 0.627 vs. 0.607, within  $1\sigma$ ): both pull the student toward the same base-shared manifold.

#### B.4.5 TRANSFER CHANNELS THROUGH DISTILLATION: LOCAL SIGNAL EXISTS, NONE SURVIVES

Beyond loss reshaping, we tested whether snapshots carry extractable transfer along three channels, plus a vision-imported learnable bridge. A zero-step diagnostic first confirmed a real *local* signal: perturbing parameters along a snapshot’s update direction predicts loss change with Pearson  $r = 0.93$  at small step sizes. None of it survives training:

A control worth recording: under the projected-bridge framework, a *random* teacher scores  $+0.014$  above the real teacher — the spectral-regularisation signature, confirming the bridge’s gain (where any) is regularisation, not transfer.

#### B.4.6 TEACHER CHOICE IN THE STREAM

Three-task stream numglue\_cm  $\rightarrow$  cstance  $\rightarrow$  numglue\_ds (target,  $N=50$ , 50/50 selected replay of cm;  $n=5$ ). Arms differ only in the KD teacher. “Forgetting” is accuracy change from the pre-target reference.

| Channel   | Method                                                  | best AVG |
|-----------|---------------------------------------------------------|----------|
| logit     | gated multi-teacher KD (top- $K$ by gradient alignment) | 0.707    |
| parameter | continuous push along aligned update directions         | 0.635    |
| feature   | relational distillation on batch similarity matrices    | 0.611    |
| feature   | learnable projected bridge ( $L = \ h_S - P(h_T)\ ^2$ ) | 0.624    |
| —         | plain cosine KD baseline                                | 0.724    |

Table 9: Transfer channels through distillation.

| teacher                                       | target acc   | cm forget | cstance forget |
|-----------------------------------------------|--------------|-----------|----------------|
| none                                          | 0.151        | +0.013    | −0.041         |
| recent (pre-task; default)                    | <b>0.167</b> | 0.000     | −0.037         |
| matched (selected source’s era)               | 0.163        | +0.004    | −0.043         |
| base (initial adapter)                        | 0.151        | −0.017    | −0.068         |
| hybrid (matched on replay / recent on target) | 0.163        | −0.009    | <b>−0.024</b>  |

Table 10: Teacher choice in the three-task stream.

The replayed task (cm) is protected in every arm — replay data itself does that work; in this *short* stream the recent snapshot is best or tied. **In the full eight-task stream the ranking reverses:** with seven candidate snapshots and substantial drift accumulated in the recent one, matched-era teachers win (TRACE-0.5B,  $n=5$ : matched  $0.458 \pm .028 >$  random  $0.445 \pm .024 >$  recent  $0.439 \pm .033$ ). The regime dependence is mechanistic: the recent snapshot is the most complete teacher but also the most drifted; as the stream lengthens, drift costs more than completeness buys. This motivates the routed-triple design of §5.

#### B.4.7 LINEAGE, NOVELTY, AND MECHANISM

**Lineage.** Distillation for stability in continual learning descends from LwF (Li & Hoiem, 2017) (previous model as teacher, matched on new-task inputs; CNN-era task-incremental classification) and iCaRL (Rebuffi et al., 2017) (exemplars plus old-class distillation). DER++ (Buzzega et al., 2020) is the closest relative: it stores each buffered sample’s logits at insertion time and anchors to them on replay — a frozen, per-sample form of era anchoring. Our era snapshots generalise this in three ways: the anchor is a queryable model rather than stored values (it can answer on any batch, including current-task batches, and is taken at task end rather than at insertion); the teacher is bound to the replay data in one routable record, so a single decision returns both; and the role assignment is inverted — DER++ points replay at stability, we point replay at transfer and give stability to distillation alone. Under our protocol the two are also empirically separated (Table 3). In recent continual learning with LLMs, distillation appears mainly as self-distillation (Yang et al., 2024); we are not aware of a system using routed, era-matched distillation as the explicit stability module.

**Mechanism: why anchoring, not knowledge, in this regime.** Three properties of the regime jointly explain the audit. (i) Ground-truth supervision is present on every batch, so teacher logits carry little marginal label information — unlike compression distillation, where the teacher is the information source. (ii) All teachers are small low-rank perturbations of one frozen base ( $r=16$ , 500 steps,  $N=50$ ), so every anchor’s pull is dominated by the shared base direction — which is why identity is second-order (E1, E4) and why deleting the shared component is catastrophic (E2). (iii) The teacher-specific component is real but *transient*: it is locally measurable (E5,  $r=0.93$ ) yet, like the one-shot parameter states of §4.2, it does not survive continued optimisation — what persists is what is re-injected identically at every step: the data, and the shared anchor. There is also a structural asymmetry: replay changes the *input* distribution, while distillation only tilts the target distribution on inputs already present — and a teacher’s source-task knowledge barely manifests on other tasks’ inputs. Cross-task distillation does carry knowledge where a jointly trained shared encoder makes the teacher signal persistent and large (multi-task vision and recommendation systems); in the carrier taxonomy of Appendix B.1 those systems occupy the persistent-parameter cell. Our negative result is scoped to the sequential, frozen-base, low-rank-snapshot regime, where we recommend assigning distillation to stability and spending the complexity budget on data selection instead.

## C EXPERIMENTAL DETAILS

### C.1 FULL-STREAM SETUP

**Configuration.** All methods train a single LoRA adapter ( $r=16$ ,  $\alpha=32$  on the  $q, k, v, o$  projections; AdamW, lr  $10^{-4}$ , batch size 4 at 0.5B / 2 at 7B, max length 384) for 500 steps per task on the  $N=50$  target examples, with a replay buffer of 100 examples per completed task ( $N_{\max}=500$  in the §4 audits). TSR:  $m=10$  probe batches, routing temperature  $\tau=0.1$ , replay ratio  $\alpha=0.5$ , distillation weight  $\lambda=0.5$  at temperature 2. The scale audit of §6.3 uses  $n=5$  seeds per scale (per-point seed s.e.m. .002–.010).

**Evaluation.** Evaluation uses 500-example per-task test sets; classification tasks report exact-match accuracy, generation tasks evaluation loss.

**Stream construction.** NumGLUE-8 arranges the eight task types of the established NumGLUE suite (Mishra et al., 2022) as a continual stream — the same construction by which Split-CIFAR derives from CIFAR and TRACE itself from eight existing datasets; two of its types (numglue-cm/ds) already appear inside TRACE.

**Cost of TSR.** Signatures are  $m=10$  forward-backward passes per task (all eight of a benchmark’s signatures take  $\sim 30$  seconds at 0.5B, and picks are scale-stable, so a small proxy model suffices); routing is an inner product; training cost matches standard replay; the buffer stores 100 examples per task ( $\sim 3$  MB for 15 tasks, versus  $\sim 130$  MB for per-task adapter libraries). We regard this routable task record — signature, data subsample, snapshot — rather than the example-level buffer of rehearsal methods or the adapter-only library of parameter-isolation methods, as the right unit of continual memory.

**New-baseline ports.** GainLoRA and DEAL (NeurIPS’25) are ported from their official releases into the stream protocol, mechanism-specific components kept as released (GainLoRA’s projection-constrained model-level gate; DEAL’s Haar-DWT retention filter) and only the adapter footprint unified for parity ( $q, k, v, o$  at  $r=16$ ,  $\alpha=32$ , lr  $10^{-4}$ ). Two adaptations: DEAL’s per-task re-initialisation depends on a checkpoint-namespace quirk that cannot restore the adapter from task 3 on, so we adopt the continuous reading consistent with the paper’s framing; and at DEAL’s native lr  $10^{-5}$  the 500-step budget under-trains in all four settings (TRACE-0.5B overall .068 vs. .372), so the standardised rate is also the charitable one.

### C.2 PER-TASK RESULTS

End-of-stream accuracy per classification task, TRACE-8 at 0.5B (canonical order, mean over 5 seeds; the per-task view behind Table 3).

| Task       | SeqFT | EWC  | LwF  | SDFT | A-GEM | O-LoRA | DER++ | ER   | TSR <sub>arg</sub> | TSR <sub>soft</sub> | TSR <sub>hyb</sub> | TSR <sub>tri</sub> |
|------------|-------|------|------|------|-------|--------|-------|------|--------------------|---------------------|--------------------|--------------------|
| C-STANCE   | .273  | .178 | .303 | .260 | .265  | .290   | .456  | .463 | .481               | .522                | .510               | .521               |
| FOMC       | .214  | .215 | .167 | .202 | .390  | .178   | .075  | .580 | .502               | .553                | .536               | .541               |
| ScienceQA  | .491  | .473 | .533 | .538 | .533  | .426   | .000  | .593 | .579               | .567                | .612               | .600               |
| NumGLUE-cm | .225  | .255 | .320 | .383 | .262  | .188   | .411  | .474 | .483               | .506                | .485               | <b>.552</b>        |
| NumGLUE-ds | .117  | .107 | .132 | .144 | .102  | .084   | .157  | .147 | .152               | .147                | .151               | <b>.159</b>        |
| <b>AVG</b> | .264  | .245 | .291 | .305 | .310  | .233   | .220  | .451 | .439               | .459                | .459               | <b>.475</b>        |

Table 11: Per-task end-of-stream accuracy on TRACE-8 at 0.5B.

Per-task tables for the remaining settings follow the same pattern and are available with the released evaluation matrices.

### C.3 STREAM ROBUSTNESS

**Task order.** Across the greedy best-source curriculum, its reversal, two deterministic reshufflings, and arbitrary reorderings, at both scales, the replay family varies by at most .03 in ranking and

magnitude; no-memory baselines vary substantially, tracking the order’s *tail* — recency, not the matrix: at 0.5B they swing by  $+.05$  to  $+.09$  under the curriculum, and an arbitrary reshuffle with a similar tail lifts SeqFT even more ( $+.12$ ); at 7B, where the base barely forgets, the same swings shrink below  $.04$ , and on NumGLUE all order effects are within seed noise (§6.4). The one mild sensitivity is argmax selection when a target precedes its best sources — the transfer matrix is directional, and soft routing removes this by construction.

**Backbone.** We further evaluate whether the framework generalises across backbone families. Following §4, we first audit headroom: on Llama-3.2-1B-Instruct (Grattafiori et al., 2024) and Gemma-3-1B-it (Gemma Team, 2025), the TRACE-8 audit finds open headroom on every task (target-full minus target-data,  $n=2$ :  $+.045$  to  $+.135$  accuracy on Llama and  $+.045$  to  $+.269$  on Gemma across the five classification targets; the three generation targets improve by 1.8–4.6 in loss), so transfer is measurable on both. Table 12 reports the stream under the exact configuration of Appendix C.1. The qualitative picture of Table 3 is unchanged: replay-based methods dominate; TSR attains the best overall accuracy, plasticity, and backward transfer on both backbones; and positive backward transfer remains confined to the replay-based methods — every rehearsal-free baseline, including the expansion-based GainLoRA, forgets.

| Method     | TRACE-8 (Llama-3.2-1B)            |                                   |                                    | TRACE-8 (Gemma-3-1B)              |                                   |                                    |
|------------|-----------------------------------|-----------------------------------|------------------------------------|-----------------------------------|-----------------------------------|------------------------------------|
|            | Overall                           | Plas                              | BWT                                | Overall                           | Plas                              | BWT                                |
| SeqFT      | $.420 \pm .028$                   | $.458 \pm .018$                   | $-.038 \pm .022$                   | $.154 \pm .018$                   | $.319 \pm .012$                   | $-.164 \pm .023$                   |
| LwF        | $.422 \pm .027$                   | $.459 \pm .014$                   | $-.038 \pm .015$                   | $.241 \pm .028$                   | $.319 \pm .012$                   | $-.078 \pm .020$                   |
| GainLoRA   | $.430 \pm .019$                   | $.438 \pm .019$                   | $-.008 \pm .009$                   | $.279 \pm .013$                   | $.301 \pm .010$                   | $-.022 \pm .012$                   |
| ER         | $.494 \pm .019$                   | $.470 \pm .012$                   | $+.024 \pm .020$                   | $.354 \pm .011$                   | $.337 \pm .012$                   | $+.018 \pm .017$                   |
| <b>TSR</b> | <b><math>.519 \pm .015</math></b> | <b><math>.481 \pm .012</math></b> | <b><math>+.037 \pm .011</math></b> | <b><math>.381 \pm .014</math></b> | <b><math>.351 \pm .012</math></b> | <b><math>+.030 \pm .010</math></b> |

Table 12: **Backbone robustness of TSR on TRACE-8** under the standard sequential protocol (Llama-3.2-1B and Gemma-3-1B; mean  $\pm$  std over 5 seeds).

#### C.4 REPRODUCIBILITY

Base models: Qwen2.5-0.5B-Instruct and Qwen2.5-7B-Instruct (Qwen Team, 2024); the backbone-robustness check additionally uses Llama-3.2-1B-Instruct (Grattafiori et al., 2024) and Gemma-3-1B-it (Gemma Team, 2025). Training configuration as in Appendix C.1 (PP15 runs use max length 256). Probe protocol, arm definitions, and per-cell seed counts as stated inline; full-seed statistics are reported throughout to avoid favourable-subset selection bias (Li et al., 2026). A minimal implementation reproducing the core TRACE-8 experiments is released at <https://github.com/ymeng3/transfer-selective-replay>.